

CSE 220 — Signals Lab

Complete Exam Preparation Guide

Python from scratch → NumPy → Matplotlib → Signal Transformations → Practice Problems

-
- Stage 0 — Python Basics (variables, lists, tuples, dicts, loops, functions, sorting)
 - Stage 1 — NumPy Fundamentals (arrays, vectorized math, indexing, aggregation)
 - Stage 2 — Matplotlib Basics (plots, subplots, styling, legends)
 - Stage 3 — Signal Transformations (time reversal, even/odd, time scaling, interpolation, sinusoids, phase vs shift)
 - Stage 4 — Worked Practice Problems (tuples/dict aggregation, matrix analysis, OOP, full subplot figure)
 - Quick-Reference Cheat Sheet

STAGE 0 — Python Basics

This stage rebuilds core Python from zero. Every construct here is something you will directly reuse in the signals labs and the practice problems.

0.1 Variables and Data Types

```
x = 5           # int
y = 3.14        # float
name = "spike"  # string
is_ready = True # boolean

print(type(x), type(y), type(name), type(is_ready))
```

0.2 Lists, Tuples, Dictionaries

List — ordered, changeable collection.

```
events = []           # empty list
events.append(("spike", 3, 3)) # add a tuple to the list
print(events[0])      # ('spike', 3, 3)
print(len(events))
```

Tuple — ordered, unchangeable. Great for fixed records like (label, amp, dur).

```
t = ("spike", 3, 3)
label, amp, dur = t      # unpacking
print(label, amp, dur)
```

Dictionary — key-value store. Perfect for grouping/summarizing by label.

```
summary = {}
label = "spike"
if label not in summary:
    summary[label] = {"total_dur": 0, "total_energy": 0}

summary[label]["total_dur"] += 3
summary[label]["total_energy"] += 3*3*3    # amp*amp*dur

for key, val in summary.items():
    print(key, val)
```

0.3 Loops and Conditionals

```
for i in range(5):
    if i % 2 == 0:
        print(i, "is even")
    else:
        print(i, "is odd")

n = 0
while n < 3:
    print("n =", n)
    n += 1
```

0.4 Functions

```
def add(a, b):
    return a + b

def describe(label, amp, dur=1):    # dur has a default value
    energy = amp*amp*dur
    return f"{label}: energy={energy}"

print(add(2, 3))
print(describe("spike", 6, 39))
```

0.5 f-strings (formatted printing)

```
label, total_dur, total_energy = "spike", 39, 351
print(f"{label} {total_dur} {total_energy}")
print(f"Average = {total_energy/total_dur:.2f}")    # 2 decimal places
```

0.6 Sorting — sorted() with a key

This is the single most useful trick for the aggregation-style problems: sort a list of tuples/dict items by any field.

```
items = [("spike", 39), ("noise", 21), ("pulse", 39)]

# Sort alphabetically by label
by_label = sorted(items, key=lambda x: x[0])

# Sort by energy descending, tie-broken by label ascending
by_energy = sorted(items, key=lambda x: (-x[1], x[0]))
print(by_energy[0])    # label with max energy, tie -> lexicographically smaller
```

Rule of thumb: **negate** a numeric field inside the key to sort it descending while keeping other fields ascending in the same call.

STAGE 1 — NumPy Fundamentals

NumPy arrays are how every signal is represented in these labs. The entire point is to avoid writing manual loops — operations apply to the whole array at once.

1.1 Creating Arrays

```
import numpy as np

t = np.arange(-5, 5, 1)      # start, stop(exclusive), step -> [-5,-4,...,4]
t2 = np.linspace(0, 1, 100) # 100 evenly spaced points from 0 to 1 (inclusive)
zeros = np.zeros(10)
ones = np.ones((3, 4))      # 3x4 matrix of ones
```

1.2 Vectorized Math (no loops needed)

```
t = np.arange(0, 10, 1)
x = np.sin(0.5 * t) + 2      # applies sin to every element at once
y = x * 3 - 1                # elementwise scale and shift
z = x + y                    # elementwise addition (arrays must match shape)
```

1.3 Indexing and Slicing

```
a = np.array([10, 20, 30, 40, 50])
print(a[0])      # 10   (first element)
print(a[-1])     # 50   (last element)
print(a[1:4])    # [20 30 40] (slice, stop exclusive)
print(a[::-1])   # [50 40 30 20 10] (reversed copy)

A = np.array([[1,2,3],[4,5,6]]) # 2D array (matrix), shape (2,3)
print(A[0, :])    # row 0 -> [1 2 3]
print(A[:, 1])    # column 1 -> [2 5]
print(A.shape)    # (2, 3) -> (rows, cols)
```

1.4 Aggregation and Search Functions

```
A = np.array([[80, 70, 90], [60, 55, 40]]) # 2 students x 3 subjects

print(np.mean(A))      # mean of ALL elements
print(np.mean(A, axis=1)) # mean of EACH ROW (per student) -> shape (2,)
print(np.mean(A, axis=0)) # mean of EACH COLUMN (per subject) -> shape (3,)
print(np.argmax(A, axis=1)) # index of best subject PER STUDENT
print(np.sum(A))
```

axis=1 collapses columns → one result per row. **axis=0** collapses rows → one result per column. Mixing these up is the most common exam mistake.

1.5 Boolean Masking (used for grade counting, thresholds)

```
P = np.array([85, 72, 55, 30, 91])
mask_A = (P >= 80)
count_A = np.sum(mask_A) # True counts as 1
grades = np.where(P >= 80, "A",
                  np.where(P >= 60, "B",
                           np.where(P >= 40, "C", "D")))
print(grades)
```

1.6 Interpolation building block: np.interp

Useful to know exists, but in the time-scaling lab you must implement interpolation manually (average of nearest left/right sample) rather than call a library function — check your assignment sheet before using this shortcut.

```
# Library shortcut (verify it's allowed before using in submission):
x_vals = np.array([0, 1, 2, 3])
y_vals = np.array([10, 20, 30, 40])
y_new = np.interp(1.5, x_vals, y_vals) # linear interpolation -> 25.0
```

STAGE 2 — Matplotlib Basics

2.1 A Single Plot with Labels and Legend

```
import matplotlib.pyplot as plt
import numpy as np

t = np.linspace(-5, 5, 200)
x = np.sin(t)

plt.figure(figsize=(6,4))
plt.plot(t, x, label="x(t)", color="blue")
plt.title("Signal x(t)")
plt.xlabel("t")
plt.ylabel("Amplitude")
plt.legend()
plt.grid(True)
plt.show()
```

2.2 Multiple Signals on One Figure

```
plt.plot(t, x, label="x(t)", color="blue")
plt.plot(t, x_even, label="xe(t)", color="green", linestyle="--")
plt.plot(t, x_odd, label="xo(t)", color="red", linestyle=":")
plt.legend()
plt.show()
```

2.3 Subplots (2x2 grid — common in exam figures)

```

fig, axs = plt.subplots(2, 2, figsize=(14, 10))

axs[0, 0].plot(months, product_a, 'b-', label="Product A")
axs[0, 0].plot(months, product_b, 'g--', label="Product B")
axs[0, 0].plot(months, product_c, 'r:', label="Product C")
axs[0, 0].set_title("Monthly Sales Trends 2024")
axs[0, 0].set_xlabel("Month"); axs[0, 0].set_ylabel("Sales (in thousands)")
axs[0, 0].legend(); axs[0, 0].grid(True)

axs[0, 1].bar(["Product A", "Product B", "Product C"],
              [product_a[-1], product_b[-1], product_c[-1]],
              color=["blue", "green", "red"])
axs[0, 1].set_title("December Sales Comparison")
axs[0, 1].grid(axis='y')

axs[1, 0].scatter(months, product_a, color="blue", s=50)
axs[1, 0].set_title("Product A Sales Distribution")
axs[1, 0].grid(True)

q_a = [sum(product_a[0:3]), sum(product_a[3:6]), sum(product_a[6:9]), sum(product_a[9:12])]
q_b = [sum(product_b[0:3]), sum(product_b[3:6]), sum(product_b[6:9]), sum(product_b[9:12])]
q_c = [sum(product_c[0:3]), sum(product_c[3:6]), sum(product_c[6:9]), sum(product_c[9:12])]
quarters = ["Q1", "Q2", "Q3", "Q4"]
axs[1, 1].bar(quarters, q_a, label="Product A", color="blue")
axs[1, 1].bar(quarters, q_b, bottom=q_a, label="Product B", color="green")
bottom_c = [a+b for a,b in zip(q_a, q_b)]
axs[1, 1].bar(quarters, q_c, bottom=bottom_c, label="Product C", color="red")
axs[1, 1].set_title("Quarterly Sales (Stacked)")
axs[1, 1].legend(); axs[1, 1].grid(axis='y')

fig.suptitle("2024 Product Sales Analysis")
plt.tight_layout()
plt.show()

```

Key API to remember: `line = ax.plot()`, `bar = ax.bar()`, `scatter = ax.scatter()`, stacked bar = repeated `ax.bar(..., bottom=previous_total)`.

STAGE 3 — Signal Transformations (Core Exam Content)

This is the heart of your three past papers (A1/A2, B1/B2, C1/C2). Master this section and you can handle any variant of these questions.

3.1 Building the Time Axis and $x(t)$

```
import numpy as np

fs = 100                                # samples per second (sampling rate)
t = np.arange(-5, 5, 1/fs)             # time axis
x = np.exp(-np.abs(t)) * np.cos(2*np.pi*t) # example base signal (given in template)
```

3.2 Time Reversal — $x(-t)$

The trick: reversing the sample ORDER of $x(t)$ gives $x(-t)$, but only if the time axis is symmetric and evenly spaced. Always verify with the flipped time axis.

```
def time_reverse(signal):
    """Returns the time-reversed version of a sampled signal."""
    return signal[::-1] # reverse the array order

x_rev = time_reverse(x) # this represents x(-t) IF t is symmetric about 0
```

If your time axis is NOT symmetric (e.g. t runs 0 to 10 only), reversing the array does not correctly align to $-t$ — you must also reverse/negate the time axis and re-check which samples land where.

3.3 Even and Odd Decomposition

Definitions: $x_e(t) = \frac{1}{2}[x(t) + x(-t)]$, $x_o(t) = \frac{1}{2}[x(t) - x(-t)]$. You must build $x(-t)$ using ONLY your `time_reverse()` function — don't recompute $x(-t)$ from the formula directly, that's the point of the assignment.

```
x_neg = time_reverse(x) # x(-t), using the required function

x_even = 0.5 * (x + x_neg) # x_e(t)
x_odd  = 0.5 * (x - x_neg) # x_o(t)

# Sanity check: x_even should look symmetric, x_odd anti-symmetric about t=0
# Also: x_even + x_odd should reconstruct x exactly
print(np.allclose(x_even + x_odd, x)) # should print True
```

Plotting requirement for this task:


```

import matplotlib.pyplot as plt

# Figure 1: x(t), xe(t), xo(t) together
plt.figure()
plt.plot(t, x, label="x(t)")
plt.plot(t, x_even, label="xe(t)")
plt.plot(t, x_odd, label="xo(t)")
plt.xlabel("t"); plt.ylabel("Amplitude"); plt.legend(); plt.grid(True)
plt.title("Even/Odd Decomposition")

# Figure 2: x(t) vs x(-t)
plt.figure()
plt.plot(t, x, label="x(t)")
plt.plot(t, x_neg, label="x(-t)")
plt.xlabel("t"); plt.ylabel("Amplitude"); plt.legend(); plt.grid(True)
plt.title("Time Reversal")
plt.show()

```

3.4 Time Sub-Scaling — $y(t) = x(t/k)$

Sub-scaling by integer $k > 1$ STRETCHES the signal in time: $y(0)=x(0)$, $y(k)=x(1)$, and everything in between ($y(1)$, $y(2)$, ..., $y(k-1)$) needs samples of x at NON-INTEGER positions that don't exist yet $\rightarrow$ must be interpolated.

```

def interpolate_signal(x, idx):
    """
    Returns an interpolated value of x at a (possibly fractional) index idx,
    using the average of the nearest left and right known samples.
    If idx lands exactly on an integer sample, return that sample directly.
    """
    left = int(np.floor(idx))
    right = int(np.ceil(idx))
    if left == right:          # idx is already an integer index
        return x[left]
    if left < 0 or right >= len(x): # out of range -> caller should discard
        return None
    return 0.5 * (x[left] + x[right])

def time_scale(x, k):
    """Returns y(t) = x(t/k) for positive integer k, using interpolation for
    the newly required in-between samples. Discards points beyond range."""
    n = len(x)
    y = []
    for i in range(n):
        idx = i / k          # fractional index into original x
        val = interpolate_signal(x, idx)
        if val is not None:
            y.append(val)
    return np.array(y)

```

Worked mini-example matching the assignment: $x = [x(0), x(1), x(2), \dots]$, $k = 2 \rightarrow y(0) = x(0)$, $y(1) = 0.5 \cdot (x(0) + x(1))$ since $\text{idx} = 0.5$, $y(2) = x(1)$ since $\text{idx} = 1.0$ exactly.

```

x_demo = np.array([0, 4, 8, 12, 16]) # toy example, x(n) = 4n
y_demo = time_scale(x_demo, 2)
print(y_demo) # [0.  2.  4.  6.  8. 10. 12. 14. 16.] (length shrinks near the tail
              # because higher indices go out of range and are discarded)

```

3.5 Discrete-Time Sinusoid: $x[n] = A \cos(\Omega_0 n + \phi)$

```
def sinusoid(n, A, Omega0, phi):
    return A * np.cos(Omega0 * n + phi)

n = np.arange(0, 50, 1)
A, Omega0, phi = 2, 0.3, 0
x_n = sinusoid(n, A, Omega0, phi)
```

3.6 Time Shift vs Phase Change

```
def time_shift_sinusoid(n, A, Omega0, phi, n0):
    """Shifts the sinusoid in time by n0 samples: x[n - n0]."""
    return A * np.cos(Omega0 * (n - n0) + phi)

def phase_change_sinusoid(n, A, Omega0, phi, delta_phi):
    """Changes only the phase term, time axis n is untouched."""
    return A * np.cos(Omega0 * n + phi + delta_phi)
```

Note the algebra: $A \cos(\Omega_0(n - n_0) + \phi) = A \cos(\Omega_0 n + (\phi - \Omega_0 n_0))$. So shifting in time by n_0 samples is EXACTLY EQUIVALENT to a phase change of $-\Omega_0 n_0$ — but only for this single-frequency sinusoid. This is why the two operations can be made to produce identical output for the right choice of $\text{delta_phi} = -\Omega_0 n_0$.

```
n0 = 5
delta_phi_equiv = -Omega0 * n0      # the matching phase change

x_shifted = time_shift_sinusoid(n, A, Omega0, phi, n0)
x_phase    = phase_change_sinusoid(n, A, Omega0, phi, delta_phi_equiv)

mse = np.mean((x_shifted - x_phase)**2)
print("MSE:", mse)    # should be ~0 when delta_phi matches -Omega0*n0

# Compare against a MISMATCHED phase change to show they normally differ:
x_phase_wrong = phase_change_sinusoid(n, A, Omega0, phi, delta_phi=0.5)
mse_wrong = np.mean((x_shifted - x_phase_wrong)**2)
print("MSE (mismatched):", mse_wrong)    # clearly larger than ~0
```

Exam talking point: time shift and phase change are related but NOT the same operation in general — they coincide only when $\phi_{\text{new}} - \phi_{\text{old}} = -\Omega_0 n_0$ exactly. For a general (non-periodic-matched, e.g. discrete non-integer-period) sinusoid, an arbitrary phase change does not correspond to any integer sample shift, and vice versa.

STAGE 4 — Worked Practice Problems

4.1 Sensor Event Aggregation (tuples + dict)

```
n = int(input())
events = []
for _ in range(n):
    label, amp, dur = input().split()
    amp, dur = int(amp), int(dur)
    events.append((label, amp, dur))

summary = {}
for label, amp, dur in events:
    if label not in summary:
        summary[label] = {"total_dur": 0, "total_energy": 0}
    summary[label]["total_dur"] += dur
    summary[label]["total_energy"] += amp * amp * dur

for label in sorted(summary.keys()):
    s = summary[label]
    print(label, s["total_dur"], s["total_energy"])

top_label = sorted(summary.keys(),
                    key=lambda L: (-summary[L]["total_energy"], L))[0]
print("TOP", top_label, summary[top_label]["total_energy"])
```

Why the sort key works: negating total_energy sorts it descending (largest first); the label L as a secondary key breaks ties alphabetically (ascending), matching "lexicographically smaller".

4.2 Student Marks Matrix Analysis (NumPy)

```

import numpy as np

A = np.array([[80, 70, 90], [60, 55, 40], [95, 88, 92]]) # S x M
max_marks = np.array([100, 100, 100])
K = 2

# Percentage matrix
P = 100 * (A / max_marks) # broadcasting divides each column correctly

# Student summary
avg = np.mean(P, axis=1) # one avg per student (collapse columns)
best_sub = np.argmax(P, axis=1) # best subject index per student

# Subject summary
sub_mean = np.mean(P, axis=0) # one mean per subject (collapse rows)
top_student = np.argmax(P, axis=0) # best student index per subject

# Best K students by average percentage
best_k_ids = np.argsort(-avg)[:K] # sort descending, take first K ids
print("Best K:", best_k_ids)

# Grade counting across the WHOLE matrix
grades = np.where(P >= 80, "A",
                  np.where(P >= 60, "B",
                           np.where(P >= 40, "C", "D")))
unique, counts = np.unique(grades, return_counts=True)
print(dict(zip(unique, counts)))

```

4.3 Student Class (OOP)

```

class Student:
    university_name = "ABC University"      # class attribute (shared by all)

    def __init__(self, name, student_id):
        self.name = name
        self.student_id = student_id
        self.grades = []                   # instance attribute (unique per object)

    def add_grade(self, grade):
        self.grades.append(grade)

    def get_average(self):
        if not self.grades:
            return 0
        return sum(self.grades) / len(self.grades)

    def get_letter_grade(self):
        avg = self.get_average()
        if avg >= 80: return "A"
        elif avg >= 60: return "B"
        elif avg >= 40: return "C"
        else: return "D"

    def __str__(self):
        return f"{self.student_id} - {self.name}: avg={self.get_average():.2f}, grade={self.get_letter_grade()}"

# Test with 5 students, 3 subjects each
students = []
for i in range(5):
    s = Student(f"Student{i+1}", i+1)
    for _ in range(3):
        s.add_grade(int(input(f"Grade for {s.name}: ")))
    students.append(s)

for s in students:
    print(s)

```

Class attribute vs instance attribute: **university_name** is defined once outside `__init__` and shared identically by every Student object. **name / student_id / grades** are set inside `__init__` with `self.`, so each object gets its own independent copy.

4.4 Full 2x2 Matplotlib Figure — See Stage 2.3 above

The complete stacked-bar / line / bar / scatter figure code is already given in Stage 2, Section 2.3 — that block is the direct answer to this exact practice problem, including `figsize=(14,10)`, `suptitle`, and `tight_layout()`.

Quick-Reference Cheat Sheet

Concept	Formula / Code
Time reversal	<code>x_rev = x[::-1]</code> (needs symmetric time axis)
Even part	<code>xe = 0.5*(x + time_reverse(x))</code>
Odd part	<code>xo = 0.5*(x - time_reverse(x))</code>
Reconstruction check	<code>xe + xo == x</code>
Time sub-scale $y(t)=x(t/k)$	<code>idx = i/k</code> ; interpolate if idx not integer
Interpolate missing sample	avg of <code>x[floor(idx)]</code> and <code>x[ceil(idx)]</code>
Discrete sinusoid	<code>x[n] = A*cos(Omega0*n + phi)</code>
Time shift	<code>x[n - n0]</code>
Phase change	<code>cos(Omega0*n + phi + delta_phi)</code>
Shift $\leftrightarrow$ phase equivalence	<code>delta_phi = -Omega0 * n0</code>
MSE between signals	<code>np.mean((x1-x2)**2)</code>
Row-wise NumPy stat	<code>np.mean(A, axis=1)</code> (per row / per student)
Column-wise NumPy stat	<code>np.mean(A, axis=0)</code> (per column / per subject)
Sort desc + tie-break asc	<code>sorted(items, key=lambda x: (-x[1], x[0]))</code>
Stacked bar	<code>ax.bar(x, seg2, bottom=seg1)</code>

Exam-day checklist: (1) time axis defined correctly (symmetric if reversal is needed), (2) required custom functions actually called (don't shortcut with formulas directly), (3) every plot has title, xlabel, ylabel, legend, grid, (4) sanity-check reconstruction identities ($x_e+x_o=x$) before submitting, (5) print statements match exact expected output format (spacing, order).